FMTH0306/Rev.1.0

Course Plan

Semester: III Year: 2025-26

Course Title: **Principles of Compiler Design**	Course Code:**25ECSC203**
Total Contact Hours: 60 Hrs	Duration of ESA:03 Hrs.
ISA Marks: 50	ESA Marks:50
Lesson Plan Authors: Prof. Nirmala Patil, Prof. Indira Bidari	Date: 18/07/2025
Checked By: Dr. Sujatha C	Date: 14/10/2025

Prerequisites: The student must be well-versed in any of the high-level programming languages (typically C language) and must be familiar with the data structures course.

Course Outcomes (COs):

At the end of the course, the student should be able to:

1. Illustrate the components of a language processing system and automata concepts in the phases of compiler design.
2. Construct an appropriate finite automaton for a given formal language.
3. Design a context-free grammar and a suitable parser for a given context-free language.
4. Perform the semantic analysis and build the intermediate code for a given programming construct.
5. Implement algorithms to demonstrate lexical and syntactical phases of a compiler.

Course Articulation Matrix: Mapping of Course Outcomes (COs) with Program Outcomes (POs)

Course Title: **Principles of Compiler Design**	Semester: III
Course Code: **25ECSC203**	Year: 2025-2026

Course Outcomes (COs) / Program Outcomes (POs)	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**
1. Illustrate the components of a language processing system and automata concepts in the phases of compiler design.	H												
2. Construct an appropriate finite automaton for a given formal language	H												
3. Design a context free grammar and suitable parser for a given context-free language.			M										
4. Perform the semantic analysis and build the intermediate code for a given programming construct.	M												
5. Implement algorithms to demonstrate lexical and syntactical phases of a compiler.					M							M	

Degree of compliance- **L**: Low **M**: Medium **H**: High

Competency addressed in the Course, corresponding Performance Indicators (PI), and Course Specific Performance Indicators (CSPI)

Competency	Performance Indicators(PI)	Course Specific Performance Indicators (CSPI)
1.1:Demonstrate the competency in Mathematics (WK2)	1.1.3:Apply concepts of mathematical structures to solve complex systems in engineering	1.1.3.1: Apply Automata theory to design and optimize finite automata from regular expressions.
		1.1.3.2: Apply rules of Context Free Grammars recursively to generate languages.
		1.1.3.3: Apply mathematical logic to design and check ambiguity of Context Free Grammar.
1.3:Demonstrate the competency in Engineering Fundamental (WK3)	1.3.2: Apply the principles of computer science including algorithms, data structures, and programming to develop a solution for complex engineering problems.	1.3.2.1: Apply knowledge of suitable data structures to store and process the symbols, machine codes, and pseudo codes in different phases of the compilation process and syntax analyzer.
		1.3.2.2: Apply machine-independent features of attribute grammars to construct the optimal semantic analyzer.
		1.3.2.3: Apply machine-independent features of the 3-address code to build intermediate code generation.
3.2: Demonstrate an ability to generate a diverse set of alternative design solutions.	3.2.2 Build models/prototypes to develop a diverse set of design solutions	3.2.2.1: Build alternative parsing algorithm for LL(1), to develop Top Down Parsers.
		3.2.2.2: Build alternative parsing algorithm for LR(0), and LR(1) to develop Bottom Up Parsers..
5.2: Demonstrate an ability to select and apply domain-specific tools, modeling techniques, and resources.	5.2.2: Apply domain-specific tools and utilize computing platforms, libraries, and resources to solve engineering problems.	5.2.2.1: Demonstrate proficiency in using LEX and YACC tools in performing lexical and syntax analysis phases of a compiler.
12.1: Demonstrate an ability to develop robust, scalable, and maintainable software systems using appropriate design principles.	12.1.1 Develop modular and reusable components using design principles.	12.1.1.1: Develop a scanner or parser using appropriate design principles.

Eg: 1.2.3.4: Represents program outcome ‘1’, competency ‘2’, performance indicator ‘3 and Course Specific Performance Indicator ‘4’’.

Course Content

Course Code: 25ECSC203	Course Title: Principles of Compiler Design	
L-T-P : 3-1-0	Credits: 04	Contact Hours/Week: 4 hrs
ISA Marks:50	ESA Marks: 50	Total Marks: 100
Teaching Hrs.: 45	Tutorial/Practical: 15 Hrs	Exam Duration: 03 Hrs.

Content	Hrs.
Unit - 1	
Chapter No.1. Introduction to Compilers Brief History of Compilers, Translation process, Major data structures in Compilers, Chomsky hierarchy, Lexical analysis: Scanning process, Regular expressions for tokens, lexical errors, applications of Regular expressions.	06 hrs.
Chapter No. 2. Finite Automata Introduction: Language, Automata, From Regular Expressions to Deterministic Finite Automata (DFA): Є-Nondeterministic Finite Automata (Є-NFA), NFA, DFA, DFA optimization, Finite automata as recognizer, Implementation of finite automata.	07 hrs.
Chapter No.3.Introduction to Syntax Analysis Introduction to Grammars, Context-Free Grammars (CFGs), Ambiguity in Grammars and Languages, Role of parsing.	05 hrs.
Unit - 2	
Chapter No. 4. Top Down Parsing Introduction, Left Recursion, Left factoring, LL (1) Parsing, FIRST and FOLLOW sets, Error recovery in Top Down Parsing.	09 hrs.
Chapter No. 5. Bottom Up Parsing Introduction, SLR (1) parsing, General LR (1) and LALR (1) Parsing, Error recovery in Bottom Up parsing.	09 hrs.
Unit - 3	
Chapter No. 6. Semantic Analysis Attributes and Attributes grammars, Algorithm for attribute computation, Symbol table, Data types and Data checking.	05 hrs.
Chapter No. 7. Intermediate Code Generation Intermediate code and data structure for code generation, Code generation of data structure references, Code generation of control statements.	04 hrs.

Text Books

1. Alfred V Aho, Monica S. Lam, Ravi Sethi, Jeffrey D Ullman, Compilers - Principles, Techniques and Tools, Updated 2nd Edition, Pearson, 2023.
2. Kenneth C Louden: Compiler Construction Principles & Practice, Cengage Learning, 1997.

References

1. Andrew W Apple, Modern Compiler Implementation in C, Cambridge University Press, 1999.
2. Charles N. Fischer, Richard J. Le Blanc, Jr, Crafting a Compiler with C, Pearson, 2011.
3. Peter Linz, An Introduction to formal languages and Automata, IV edition, Narosa, 2016.
4. Basavaraj S Anami, Karibasappa K.G, Formal Languages and Automata Theory, First, Wiley India, 2011.

Evaluation Scheme

ISA Scheme

Assessment	Weightage in Marks
In Semester Assessment -1 (ISA1)	15
In Semester Assessment -2 (ISA2)	15
Self Learning	15
Team Activity	05
Total	50

Course Unitization for ISA and ESA

Topics / Chapters	Teaching Credits	No. of Questions in ISA-1	No. of Questions in ISA-2	No. of Questions in Activity (Tutorials)	No. of Questions in ESA
Unit I					
1. Introduction to Compilers	06	1.00	--		1.00
2. Finite Automata	06	1.00	--	5	1.00
3. Introduction to Syntax Analysis	04	1.00	--	5	1.00
Unit II					
4. Top Down Parsing	08	--	1.5	5	1.5
5. Bottom Up Parsing	08	--	1.5	5	1.5
Unit III					
6. Semantic Analysis	04			3	1.00
7. Intermediate Code Generation	04			3	1.00

Note

1. Each Question carries 20 marks and may consists of sub-questions.
2. Mixing of sub-questions from different chapters within a unit (only for Unit I and Unit II) is allowed in ISA I, II and ESA
3. Answer 5 full questions of 20 marks each (two full questions from Unit I, II and one full questions from Unit III) out of 8 questions in ESA.

Course Assessment Plan

Course Title: Principles of Compiler Design					Code: 25ECSC203
Course outcomes (COs)	Weightage in assessment	Assessment Methods			
		ISA – I	ISA - II	Tutorial	ESA
1. Illustrate the components of a language processing system and automata concepts in the phases of compiler design.	15%	✓			✓
2. Construct an appropriate finite automaton for a given formal language.	15%	✓		✓	✓
3. Design a context free grammar and suitable parser for a given context-free language.	40%	✓	✓	✓	✓
4. Perform the semantic analysis and build the intermediate code for a given programming construct.	10%				✓
5. Implement algorithms to demonstrate lexical and syntactical phases of a compiler.	20%			✓	
Weightage		20%	20%	20%	40%

Course Activity and Rubrics

Tutorial Schedule

Expt. /Job No	Brief description of experiments	No of slots
1.	Introduction, Regular expressions.	02
2.	NFA, DFA, and DFA optimization.	02
3.	Designing context-free grammars.	01
4.	Implementation of a lexical analyzer using the LEX tool.	02
5.	Implementation of a syntax analyzer using the YACC tool.	02
6.	Top-down parsing.	02
7.	Bottom-up parsing.	02
8.	Development of a compiler component.	02

List of Lex and YACC Experiments:

Sl. No	Experiments
1.	Write a program in LEX to recognize different tokens: Keywords, Identifiers, Constants, Operators, and Punctuation symbols.
2.	Design and implement programs to recognize various strings given in the form of a regular expression
3.	Write a program in LEX to recognize Floating Point Numbers
4.	Write a LEX program to recognize the following tokens over the alphabet {0,1,..,9} a. The set of all strings ending in 00. b. The set of all strings with three consecutive 222's.
5.	Write a LEX program to recognize : a. PAN number b. Aadhaar number c. Vehicle number
6.	Write a Lex and YACC program to implement, validate, and evaluate an arithmetic expression.
7.	Write a Lex and YACC program to implement, validate, and evaluate nested if statements.
8.	Write a Lex and YACC program to validate the following using context context-free grammar : a. Equal number of 0's and 1's b. Palindrome on a set T={a,b}
9.	Write a Lex and YACC program for the following recursive descent CFG: S → aaSaa | aa S → aAd | bBd | aBe | bAe A → c B → c
10.	Development of scanner or parser for the given grammar.

Self Learning

L	T	P	Project	Credit (L+T+P)	Theory	Tutorial	Lab.	Project	Sub Total	Self-Learning	Grand Total
					Hrs	Hrs	Hrs	Hrs	Hrs	Hrs	Hrs
3	1	0	0	4	45	15	0	0	60	60	**120**

Sl. No	Activity	Objective	Self-learning Hours	Assessment	Weightage in %	Deliverable / Evidences
1	**Concept Based Post test** **Compiler Foundations Checkpoint.** The topics include: • Fundamentals of Computer Organization Architecture: memory hierarchy, addressing modes, instructions etc.) (LMS) • C Programming: declaration, looping, decision making constructs, functions, file handling. (LMS) • Data Structures: trees, stack, hash table. (LMS)	To assess the understanding of the prerequisites required for developing the analysis phases of a compiler. **Chapter 1 to 5** **BL: 2** **CO1** **PI: 1.3.2**	10	Post test On LMS	20%	Post test questions and Answers Division wise Marks (LMS/Documentation)
2	**Tool Learning: Lex** To explore the Lexical analysis generator tool (Lex). The topics include: • Lex specification • Predefined variable and functions. • Meta-characters • Commands to Execute Lex program. https://www.youtube.com/watch?v=s2AfL99gKwU&list=PLEAYkSg4uSQ3yc_zf_f1GOxI5CZo0LVBb&index=8	To design regular expressions, implement Lex programs, and validate the functional correctness of the lexical analyser. **Chapter 1 and 2** **BL:3** **CO2 and CO3** **PI: 1.1.3** **5.2.2**	25	Implementation of Lex program for given requirements .	30%	Learning Portfolio

3	**Tool Learning: YACC** To explore the syntax analysis generator tool (YACC). The topics include: • YACC specification • Predefined variable and functions. • Context free grammar. • Commands to Execute YACC program. https://www.youtube.com/watch?v=z87PPnwtjKE&list=PLEAYkSg4uSQ3yc_zf_f1GOxl5CZo0LVBb&index=21	To design a context-free grammar for a given language specification, implement its syntax analyser using YACC, and validate the correctness of its parsing functionalities. **Chapter 3, 4 and 5** **BL:3** **CO2 and CO3** **PI: 3.2.2** **5.2.2**	25	Implementation of YACC program for given requirements .	30%	Learning Portfolio
			Total Self Learning hrs: 60		**Total marks: 80 Scaled to 15 marks**	
Final Document to be submitted to department and stored in DMS.						

Team activity

Activity	**Objective**	**Assessment**	**Weightage in %**	**Deliverable / Evidences**
Construction of Scanner and Parser To build the scanner and parser using LEX and YACC tools	To design regular expression and context-free grammar for a given language specification, implement lexical and syntax analysers using LEX &YACC tool. **Chapter 1 to 5** **BL: 3** **CO5** **PIs: 12.1.1**	Implementation of scanner and parser for given requirements.	20%	Learning Portfolio
			Total marks are 20. Scaled to 05	

Assessment parameters and Rubrics

Parameter	Excellent	Good	Satisfactory	Needs improvement
1. Problem Analysis and Solution **PI: 1.1.3, 3.2.2**	• Identify suitable automata concepts. • Design a regular expression and an optimized DFA to solve the problem. • Design and provide alternative optimized solutions.	• Identify automata concepts. • Design a regular expression and a DFA to solve the problem. • Design and provide alternative solutions.	• Identify automata concepts. • Design a regular expression to solve the problem. • Design and provide a solution.	• Identify automata concepts. • Unable to design a regular expression to solve the problem. • Unable to provide suitable design solutions.
2. Program Execution **(LEX & YACC programming)** **PI: 5.2.2**	Develop LEX and YACC programs with precise regular expressions and CFG.	Develop LEX and YACC programs with regular expressions and CFG.	Develop LEX and YACC programs with regular expressions	Unable to write correct LEX and YACC programs.
Team activity: Building Scanner and Parser				
3. Development of compiler component. (10 Marks) **(C programming) and Coding standards.** **PI: 12.1.1**	• Student is able to build a scanner/parser for a given language construct. • Implement the functionalities of scanner/parser using modular programming and validate for all the test cases.	• Student is able to build a scanner/parser for a given language construct. • Implement the few functionalities of scanner/parser using modular programming and validate for a few test cases.	• Student is able to build a scanner/parser for a given language construct. • Implement the few functionalities of scanner/parser without using modular programming and validate for a few test cases.	Unable to implement the basic functionalities of the scanner/parser.

Date: 14-10-2025 **Head, CSE**

Chapter wise Plan

Course Code and Title: **25ECSC203/Principles of Compiler Design**	
Chapter Number and Title: **Chapter No.1. Introduction to Compilers**	Planned Hours:**06 hrs.**

Learning Outcomes:-

At the end of the topic the student should be able to:

Sr.No	TLO's	CO's	BL	CA Code
1.	Explain the different phases of a compiler.	CO1	L2	1.3
2.	Explain the different data structures of a compiler.	CO1	L1	1.3
3.	Demonstrate the functions of lexical analyzer.	CO1	L3	1.3
4.	Explain the classification of formal languages.	CO2	L2	1.3
5.	Build the regular expression for programming constructs.	CO2	L3	1.3

Lesson Schedule Class No. - Portion covered per hour
1. Brief History of Compilers
2. Translation process
3. Major data structures in Compilers, Chomsky hierarchy
4. Lexical analysis: Scanning process
5. Regular expressions for tokens, Lexical errors
6. Applications of Regular expressions

Review Questions

Sr.No. - Questions	TLO	BL	PI Code
1. Explain the different phases and the structure of a compiler.	TLO1	L2	1.3.2
2. What are some examples of modern compilers, and how do they differ from early compilers?	TLO1	L2	1.3.2
3. What are the main data structures used in compilers, and why are they important?	TLO2	L2	1.3.2
4. How does the Chomsky hierarchy classify formal languages?	TLO2	L2	1.3.2
5. How can regular expressions be used to validate email addresses or phone numbers?	TLO5	L3	1.3.2
6. How does the lexical analyzer report the error for the following statement? A = b + @3; Which are those errors? Justify your answer	TLO3	L3	1.3.2

7. Identify the content of Symbol table at the end of lexical analysis process for the statement a=b+c*100.	TLO2	L3	1.3.2

8. Illustrate each phase of compiler with an example. For the given C program, identify the lexemes and their token type. Count the number of valid tokens.
int main ( )
{
int *a, b;
b=&10;
a=&b;
printf("%d%d", b, a);
b=* b; /*pointer*/
}
	TLO2	L3	1.3.2

10.Explain the role of lexical analyzer with a neat diagram. What are the contents of symbol table and literal table for the code snippet provided in Q.4.	TLO3	L3	1.3.2

10. Identify the tokens for the given C program below:
int main()
{
int a,b c;
a=10;b=5;
c=a+b;
printf("value of c is:",c);
return 0;
}
	TLO2	L3	1.3.2

11. Design the regular expression for the following regular languages: i. A string of 0's and 1's with no consecutive 0's in it. ii. A string of 0's and 1's ending with 001. iii. Ending with either 00 or 11. iv. 5th symbol from right end is 1.	TLO5	L3	1.3.2
12. Design regular Expressions for the following languages over the alphabets {a, b}. i. String should start and end with different symbol. ii. String should be odd in length and starts with a. iii. String length should be divisible by 3. iv. String has substring abaa maximum 3 times.	TLO5	L3	1.3.2
13.Design a regular expression for an URL with the following constraints: i. Must start with http or https or ftp followed by :// ii. Must match a valid domain name iii. Could contain a port specification (http://www.sitepoint.com:80) iv. Could contain digit, letter, dots, hyphens, forward slashes, multiple times	TLO5	L3	1.3.2
14. Design a regular expression for the following i. To accept signed or unsigned decimal numbers with optional exponent ii. To accept the language L= $\{a^n b^m \mid m+n \text{ is even}\}$ iii. To accept the student registration number (SRN) for the students belonging to Computer science, Mechanical and Civil branches in KLE Technological University (Make suitable assumptions if required).	TLO5	L3	1.3.2

Course Code and Title: **25ECSC203 / Principles of Compiler Design**	
Chapter Number and Title: **Chapter No.2. Finite Automata**	Planned Hours: **07 hrs.**

Learning Outcomes:

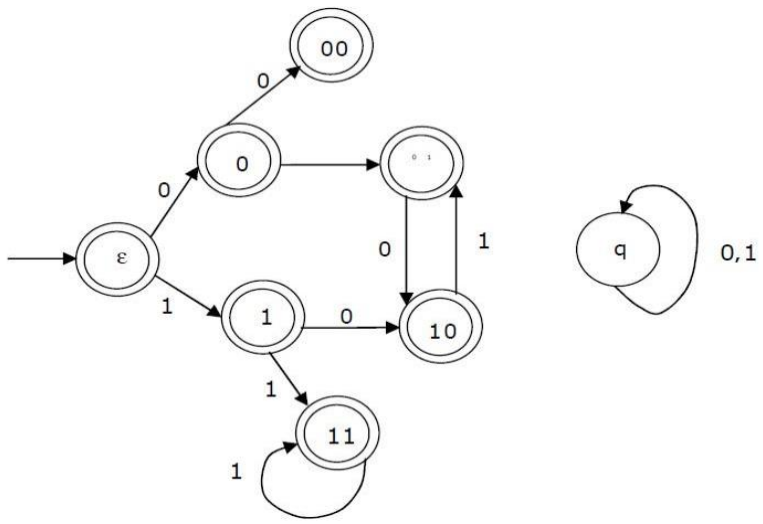
At the end of the topic the student should be able to:

Sr.No	TLO's	CO's	BL	CA Code
1.	List the different representations of formal languages	CO1	L2	1.3
2.	Design a DFA for the given regular languages	CO2	L3	1.1
3.	Design a NFA and NFA's with Є-transitions for a regular language	CO2	L3	1.1
4.	Determine the equivalence between DFA, NFA and Є-NFA	CO2	L3	1.1

Lesson Schedule Class No. - Portion covered per hour
1. Introduction: Language, Automata, Equivalence of Automata
2. From regular expressions to Є-Nondeterministic Finite Automata (Є-NFA)
3. From Є-NFA to Nondeterministic Finite Automata(NFA)
4. From NFA to Deterministic Finite Automata (DFA)
5. Solving problems on DFA
6. Optimization of Deterministic Finite Automata
7. Finite automata as recognizer, Implementation of Finite Automata

Review Questions

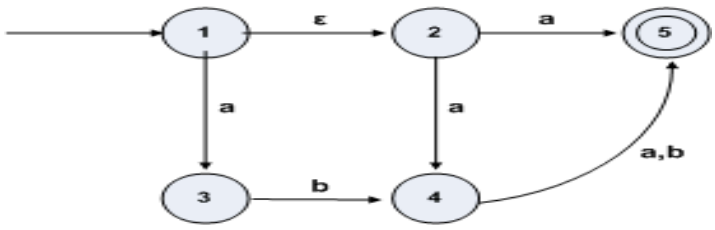
Sr.No. - Questions	TLO	BL	PI Code
1. Classify the following formal languages into different classes with justifications I) unsigned numbers II) Valid parenthesized expressions	TLO1	L2	1.3.2
2. Design the efficient automata for ab*c|d. Justify your answer.	TLO2	L3	1.1.3
3. How is a regular expression initially converted into an NFA using Thompson's construction?	TLO4	L3	1.1.3
4. How are union, concatenation, and Kleene star operations represented in an NFA?	TLO3	L2	1.1.3
5. How are states in the DFA derived from the states of the NFA during the conversion process?	TLO4	L3	1.1.3

6. What are the differences between a DFA and an NFA in the context of regular expressions?	TLO4	L3	1.1.3
7. Construct a Nondeterministic Finite Automata (NFA) for the Regular Language where string will be accepted if it ends with aa. Later convert the constructed Nondeterministic Finite Automata (NFA) to equivalent Deterministic Finite Automata (DFA).	TLO3	L3	1.1.3
8. Design a Є-NFA using Thompson's construction method to accept strings that begins with ab over the Σ={a,b}* and convert it to Deterministic Finite Automata using the appropriate method.	TLO3	L3	1.1.3
9. Consider the set of strings on {0,1} in which, every substring of 3 symbols has at most two zeros. For example, 001110 and 011001 are in the language, but 100010 is not. All strings of length less than 3 are also in the language. A partially completed DFA that accepts this language is shown below. Find the missing arcs in the below DFA to accept all the strings accepted by the language. 	TLO2	L3	1.1.3
10. Design a DFA such that:L = {$a^n b^m c^l$ | n,m,l ≥ 1} for a given input alphabet, Σ={a, b, c}	TLO2	L3	1.1.3

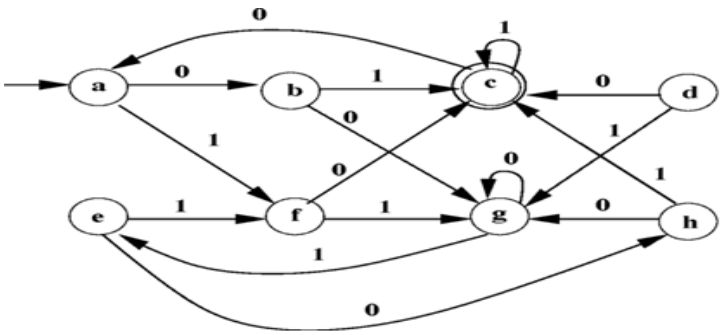
<table>
<tr><td>11. Verify whether the following State Transition Table which represents Deterministic Finite Automata (DFA),Give a regular expression for the language of the automation.

<table>
<tr><th></th><th>0</th><th>1</th></tr>
<tr><td>->q1</td><td>q2</td><td>q3</td></tr>
<tr><td>q2</td><td>q1</td><td>q3</td></tr>
<tr><td>q3*</td><td>q2</td><td>q1</td></tr>
</table>

</td><td>TLO4</td><td>L3</td><td>1.1.3</td></tr>
<tr><td>12. Convert the following NFA to DFA.



</td><td>TLO4</td><td>L3</td><td>1.1.3</td></tr>
<tr><td>13. For the given Deterministic Finite Automata (DFA), find the indistinguishable, distinguishable states and draw the optimized DFA.



</td><td>TLO4</td><td>L3</td><td>1.1.3</td></tr>
</table>

Course Code and Title:**25ECSC203/ Principles of Compiler Design**	
Chapter Number and Title: **Chapter No.3. Introduction to Syntax Analysis**	Planned Hours: **05 hrs.**

Learning Outcomes:

At the end of the topic the student should be able to:

Sr. No	TLO's	CO's	BL	CA Code
1.	Explain Context free grammar and parsing.	CO3	L2	1.1
2.	Design the context free grammar for the given language.	CO3	L3	3.2
3.	Identify and describe the ambiguity in the context free language.	CO3	L2	1.1
4.	Identify errors and recovery strategies	CO3	L2	1.1

Lesson Schedule Class No. – Portion covered per hour
1. Role of parser, Regular Grammars
2. Introduction to Design Context-Free Grammars (CFGs)
3. Exercises on Designing CFGs
4. Parsing and ambiguity in Grammars and Languages
5. Different errors and recovery strategies

Review Questions

Sr.No. – Questions	TLO	BL	PI Code
1. Identify the language of grammar G; S → AS | Є; A →A1 | 0A1 | 01 is ambiguous or unambiguous. If it is ambiguous identify the alternative grammar.	TLO3	L3	1.1.3
2. Show that the following language is context free: L={ $uvwv^r$: u, v, wϵ $\{a, b\}^+$, |u|=|w|=2}	TLO2	L3	3.2.2
3. Construct a grammar to accept the following language: L={w ,|w| mod 3>0 where w ϵ $\{a\}^*$ }	TLO2	L3	3.2.2
4. Design a context free grammar for syntactically correct infix algebraic expression using terminals +, *, (,),- .x, y and z.	TLO2	L3	3.2.2

<table>
<tr><td>5. A student wrote two context-free grammars G1 and G2 for generating a single C-like array declaration. The dimension of the array is at least one.
For example, int a[10][3];
The grammars use D as the start symbol, and use six terminal symbols: int ; id [ ] num

<table><tr><th>Grammar G1</th><th>Grammar G2</th></tr><tr><td>D → int L;
L → id [E
E → num]
E → num] [E</td><td>D → int L;
L → id E
E → E[num]</td></tr></table>
Which of the grammars correctly generate the declaration mentioned above?
(a) Both G1 and G2 (b) Only G1 (c) Only G2 (d) Neither G1 nor G2</td><td>TLO1</td><td>L3</td><td>1.1.3</td></tr>
<tr><td>6. Design a grammar that generates same language as the following regular expression: (a | c | ba | bc) * (b| ϵ)</td><td>TLO2</td><td>L3</td><td>3.2.2</td></tr>
<tr><td>7. Design grammar G to accept strings starting with at least two 0’s and ending with at least two 1’s, over the alphabet Σ= {0, 1}.</td><td>TLO2</td><td>L3</td><td>3.2.2</td></tr>
<tr><td>8. List different error recovery strategies and explain panic mode error recovery.</td><td>TLO4</td><td>L2</td><td>1.1.3</td></tr>
<tr><td>9. Consider the following grammar.
E→ E+E
E→E-E
E→ E*E
E→ €|num
For the string (8+6*(8-3)) show the grammar is ambiguous using leftmost derivation and its parse tree.</td><td>TLO3</td><td>L3</td><td>1.1.3</td></tr>
<tr><td>10. Give the salient features of CFG with an example.</td><td>TLO1</td><td>L2</td><td>1.1.3</td></tr>
</table>

Course Code and Title: **25ECSC203/ Principles of Compiler Design**	
Chapter Number and Title: **Chapter No.4.Top Down Parsing**	Planned Hours: **09 hrs.**

Learning Outcomes:

At the end of the topic the student should be able to:

Sr.No	TLO's	CO's	BL	CA Code
1.	Explain the role of the parser in the compiler model.	CO1	L2	1.1
2.	Differentiate lexical analyzer and syntactic analyzer	CO1	L2	1.1
3.	Identify whether the given grammar is suitable for Top-Down parsing.	CO3	L3	3.2
4.	Compute the FIRST and FOLLOW for the given grammar.	CO2	L3	1.1
5.	Describe LL (1) grammar.	CO3	L2	1.1
6.	Construct the predictive parsing table for a given grammar.	CO3	L3	3.2

Lesson Schedule Class No. – Portion covered per hour
1. Top Down parsing
2. Left Recursion
3. Left factoring
4.Exercises on Left factoring
5. Introduction to LL (1) Parsing
6. Constructing LL (1) Parser
7. LL1-FIRST sets
s8. LL1-FOLLOW sets
9. Error recovery in Top Down Parsing

Review Questions

Sr.No. – Questions	TLO	BL	PI Code
1. Define Top-down parsing and explain error recovery actions in LL(1) parser with an example.	TLO1	L2	1.3.2
2. Explain the role of parser during the compilation.	TLO1	L2	1.3.2
3. Compare the salient features of lexical analyzer and syntactic analyzer.	TLO2	L2	1.3.2
4. Define Left factoring. Apply left factoring for the grammar: S→ bSSaaS | bSSaSb | bSb | a	TLO3	L2	1.1.3
5. Eliminate left recursion from the grammar	TLO3	L2	1.1.3

S → aB | aC | Sd | Se | a B → bBc | f C → g Identify whether the resulting grammar is left factored or not. If not, left factor the grammar.			
6. Define left recursion. Identify whether the given grammar is suitable for Top-down parsing E → E + T | F, T →T * F | F, F → (E) |	TLO3	L2	1.1.3
7. Construct the Predictive parser for the following programming construct **if i> j then i: = I – j else j: = j – I;**	TLO6	L3	3.2.2
8. Construct the top-down parsing table for the grammar: E→E+T | T T→ id + T | id	TLO4	L3	3.2.2
9. Consider the following grammar over the alphabet {c, d, e}: S →A B A A →B c | d A | ϵ B →e A Identify First and Follow sets. Construct LL (1) parsing table. Is this LL (1) grammar. Justify	TLO4 TLO5	L3	3.2.2
10. Consider the following grammar G: Answer the following. S → Sa | bL L → ScL | Sd | a i. Is the grammar is ambiguous and is it suitable for LL (1) parsing, if not rewrite the grammar so that it can be parsed by LL (1). ii. Identify First and Follow. Construct the parse table. iii. Show a top-down parse of the string bbacbada	TLO4 TLO5 TLO6	L3	3.2.2
11. Consider the grammar S→ A|a, A→a . Is the grammar is LL (1).	TLO6	L3	1.1.3
12. Consider the grammar G={V, T, P, S} where V={E, T, X, Y} and T={(,), +, *, int} P : E→ TX T→ (E) | int y X → +E | e Y → *T | e Perform all the steps of LL(1) parser and show the contents of parsing stack to parse the string: (int * int)	TLO3 TLO4 TLO6	L3	3.2.2
13. Why do we apply left factoring on a grammar. Apply left factoring to the following grammar. P → r P P s s P | r P P s P r | s P s | r | r X X → p q X | p X q | p q r X | p	TLO3	L3	3.2.2
14. What is the need to eliminate left recursion from the grammar in top down parsers. Justify if the given grammar is LL(1) or not. If not rewrite the grammar, and compute the first and follow sets for the grammar. S → A B A → C a | ϵ B → B a A C | c C → b | ϵ	TLO5	L3	1.1.3

Course Code and Title: **25ECSC203/ Principles of Compiler Design**	
Chapter Number and Title: **Chapter No.5. Bottom up Parsing**	Planned Hours: **09 hrs.**

Learning Outcomes:

At the end of the topic the student should be able to:

Sr.No	TLO's	CO's	BL	CA Code
1.	Differentiate Top-Down and Bottom-up parsing	CO3	L2	1.1
2.	Construct SLR parsing table for a given grammar	CO3	L3	3.2
3.	Apply the LR-parsing techniques to construct the SLR, LR (1) and LALR parsing tables from the given augmented grammars	CO3	L3	3.2
4.	Differentiate LR and LALR parsing	CO3	L2	3.2
5.	Construct LALR parsing tables for a given grammar	CO3	L3	3.2

Lesson Schedule Class No. – Portion covered per hour
1. Bottom-Up parsing
2. Overview
3. LR (0) parsing
4. SLR (1) parsing
5. Exercises on Shift/Reduce conflicts
6.CLR(1)
7. LALR (1) Parsing
8. Exercises on Shift/Reduce conflicts
9. Error recovery in bottom-up parsing

Review Questions

Sr.No. - Questions	TLO	BL	PI Code
1. Differentiate the functions of Top Down and Bottom-up parsers.	TLO1	L2	1.1.3
2. Compare LL(1) parser with LR parser.	TLO1	L2	1.1.3
3. Briefly explain the requirement of LR Parser.	TLO2	L2	1.1.3
4. Design the appropriate parser for the grammar: S→ (E) b| a E→E, S | S Provide the sentence which satisfies the condition of the grammar defined above.	TLO3	L3	3.2.2

5. Construct LR(0),SLR (1) parsing table for the grammar: S →Ba, B→ bC | D, C→ aaC | c, E →ab. Check the existence of alternative parser for the above grammar? If yes design the alternative parser for the same.	TLO2	L3	3.2.2
6. Consider the following grammar; S → Z Z → aMa | bMb | aRb | bRa M → c R → c Is it SLR (1) and LALR (1)? Explain.	TLO5	L3	1.1.3
7. Explain why bottom-up parsers result in right-most derivations in reverse.	TLO1	L2	1.1.3
8. Describe the difference between bottom-up and top-down parsing with respect to a grammar production A →β	TLO1	L2	1.1.3
9. Consider the grammar: S →Sa | Sc | c i. Compute the canonical set of LR (1) items for the grammar. ii. Build the ACTION/GOTO table for the grammar. iii. Explain why the grammar is or is not LR (1)	TLO3	L3	3.2.2
10. Consider the following grammar: S → aAd | bBd | aBe | bAe A → c B → c Differentiate between LR (1) and LALR (1) parser using above grammar.	TLO4 TLO5	L3	3.2.2
11. Consider the grammar: S → AaAb | BbBa | ϵ | ϵ .Check whether the grammar is LR(0), SLR(1),CLR(1), LALR(1) ?	TLO3	L3	1.1.3
12. Consider the following grammar: S → Aa | BAc | dc | bda, A→d. Check whether the grammar is SLR (1), LR (1) ?	TLO3	L3	3.2.2
13. How do we say that for a grammar G belongs to the LR parsers or not. Give examples.	TLO2	L2	3.2.2
14. Show the following grammar is SLR(1) grammar or not with the help of DFA of LR(0) items . S→ AS | b A→ SA | a Justify your answer?	TLO5	L3	3.2.2
15. Differentiate between LR(0), LR (1) and SLR (1), LL(1) and CLR(1) parser with the help of suitable grammar.	TLO4	L3	1.1.3
16. Determine whether the following grammar is LL(1) and SLR(1). S → d A | a B A → b A | c B → b B | c	TLO5	L3	3.2.2

Chapter Number and Title: **Chapter No.6.Semantic Analysis**	Planned Hours: **05 hrs.**

Learning Outcomes:

At the end of the topic the student should be able to:

Sr.No	TLO's	CO's	BL	CA Code
1.	Identify attributes of language entities for the computation	CO4	L3	1.3
2.	Define a data type attribute for the variables given by the identifiers in a declaration	CO4	L1	1.3
3.	Write attribute equations or semantic rules related to the grammar of the language	CO4	L3	1.3
4.	Draw a parse tree showing attribute computations for the given grammar.	CO4	L3	1.3

Lesson Schedule Class No. - Portion covered per hour
1. Attributes and Attributes grammars
2. Algorithm for attribute computation
3. Attribute Computation Contd.
4. Symbol table
5. Data types and data checking

Review Questions

Sr.No. - Questions	TLO	BL	PI Code
1. Consider the following simple grammar of variable declarations in a C-like syntax: D → T V T →i | f V→ a, V |b List the attributes associated with the above grammar and the order of evaluation.	TLO3	L3	1.3.2
2. Design the attribute grammar for the following grammar G. decl→ type var-list type → **int** | **float** var-list → **id,** var-list | **id** And also show the annotated parse tree for the string : **int a, b, c**	TLO4	L3	1.3.2
3. Explain the synthesized and inherited attributes with examples.	TLO1	L2	1.3.2

4. Describe the role of semantic analysis in the compilation process. How does it differ from syntax analysis?	TLO1	L2	1.3.2
5. What are attributes in the context of compiler design, and how are they classified?	TLO1	L3	1.3.2
6. What is an attribute dependency graph, and why is it important in evaluating attribute grammars?	TLO4	L2	1.3.2
7. Explain the differences between L-attributed and S-attributed grammars. Provide examples where each might be preferred.	TLO3	L3	1.3.2
8. Explain the role of attribute grammars in optimizing compilers, specifically in the context of data flow analysis and optimization.	TLO3	L2	1.3.2
9. Given the grammar for simple arithmetic expressions: $E \rightarrow E + T \mid T$ $T \rightarrow T * F \mid F$ $F \rightarrow (E) \mid num$ Assume num is an integer. i. Define synthesized attributes for calculating the value of expressions represented by E, T, and F. ii. Write the semantic rules for each production to compute the value of the expression.	TLO1	L3	1.3.2

Course Code and Title: **25ECSC203/ Principles of Compiler Design**	
Chapter Number and Title: **Chapter No.7. Intermediate Code Generation**	Planned Hours: **04 hrs.**

Learning Outcomes:

At the end of the topic the student should be able to:

Sr.No	TLO's	CO's	BL	CA Code
1.	Explain the intermediate code representations	CO4	L2	1.3
2.	Translate given expression to three address or P code	CO4	L3	1.3
3.	Describe the data structures used for implementation of 3 address code	CO4	L2	1.3
4.	Sketch the code arrangement for the control/logical statement	CO4	L3	1.3
5.	Generate 3 address code for control and logical statements	CO4	L3	1.3

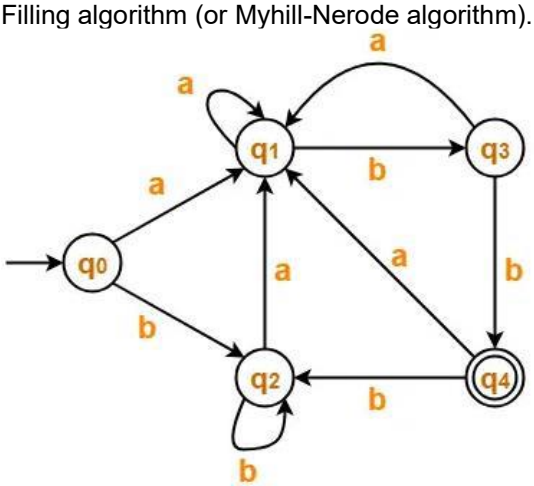
Lesson Schedule Class No. - Portion covered per hour

1. Intermediate code and data structures for code generation
2. Code generation of data structure references
3. Code generation of control statements
4. Code generation of control expressions

Review Questions

Sr.No. - Questions	TLO	BL	PI Code
1. Represent the statement given below in 3 address code.2*a+(b-3).	TLO2	L3	1.3.2
2. Explain the code generation of data structure references.	TLO5	L2	1.3.2
3. Explain the code generation for a while statement.	TLO4	L2	1.3.2
4. Generate the 3 address code for any two logical statements	TLO2	L2	1.3.2
5. Convert the expression x = (y + z) * (w - v) into P-code, assuming all variables are already declared.	TLO2	L3	1.3.2
6. Given the conditional expression if (a < b) then c = d else c = e, show how it would be represented in three-address code.	TLO5	L3	1.3.2
7. Translate the conditional expression if x > y goto L1; else goto L2 into P-code.	TLO5	L3	1.3.2

Course Code: 25ECSC203	Course Title: Principles of Compiler Design
Duration:1 hr. 15 min	Max. Marks:40
Note: **Answer Any Two Full Questions**	

Q.No	Questions	Marks	CO	BL	PI Code
1.a	Design a regular expression for the following with the given constraints and list at least 2 valid lexemes for each. i. Validate a Credit Card Number (Visa, MasterCard, Amex) A credit card number should: Contain 16 digits (Visa/MasterCard) or 15 digits (Amex) Be grouped in sets of 4 digits separated by spaces ii. **L= {a^{2m} b^{2m+1} where m>=1}**	8	CO1	L2	1.3.2
1.b	What are the different types of automata? Design a DFA to accept the strings starting with at least two 0's and ending with at least two 1's, over the alphabet Σ= {0, 1}.	6	CO2	L3	1.1.3
1.c	Consider the following grammar, aS | aSbS | Є◊S Find leftmost, rightmost derivation and parse trees for the string "aab". Check whether the grammar is ambiguous	6	CO3	L3	1.1.3
2.a	Design a Є-NFA for the regular expression **(a|b)abb*** using Thompson's construction method and list any four strings that belong to language.	8	CO2	L3	1.1.3
2.b	Illustrate the different phases in translation process of a source code into target code with an example.	6	CO1	L2	1.3.2
2.c	Design the Context Free Grammar to generate the language: **L = {$0^m1^m2^n$ | m>=1, n >= 0}**	6	CO2	L3	1.1.3
3.a	Optimize the given Deterministic Finite Automata (DFA), using Table Filling algorithm (or Myhill-Nerode algorithm).  	8	CO2	L3	1.1.3

3.b	Convert the following NFA to DFA using subset construction method: δ | 0 | 1 → p | {p, q} | {q} *q | {r} | {q, r} r | {s} | {p} *s | ϕ | {p}	6	CO1	L3	1.3.2
3.c	Describe the Chomsky hierarchy and illustrate its classification with a neat diagram.	6	CO3	L2	1.1.3

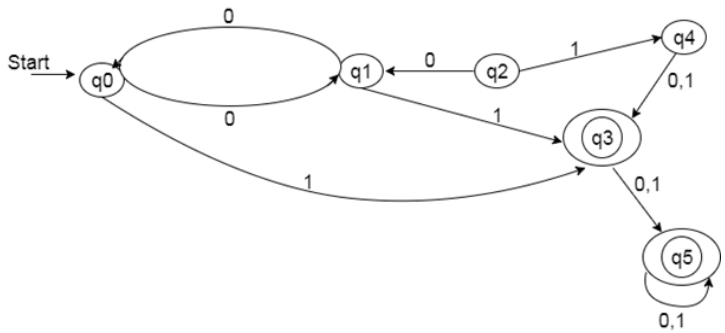
Model Question Paper for Minor Examination (ISA-2)					
Course Code: 25ECSC203		Course Title: Principles of Compiler Design			
Duration:1 hr. 15 min		Max. Marks:40			
Note: **Answer Any Two Full Questions**					
Q.No	Questions	Marks	CO	BL	PI Code
1.a	Consider the following grammar and apply LR(1) algorithm: S → AaAb | BbBa A → ε B → ε i. Construct corresponding DFA. ii. Build Parse table. Process the input string **ab** using stack.	8	CO3	L3	3.2.2
b.	Consider the grammar: St_seq→Stmt St_seq1 St_seq1 →; St_seq | ε Stmt→ s i. Apply the error recovery process of LL(1) to fill the empty cells in the parse table given below: ; | s | $ St_seq | | Stmt→ St_seq1 | St_seq1 | St_seq1→; St_seq | | St_seq1 → ε Stmt | | Stmt→ s | ii. Demonstrate the error recovery process during the parsing	6	CO3	L3	1.1.3
c.	Answer the following i. Which grammars are suitable for top-down parsing?	6	CO1	L2	1.1.3

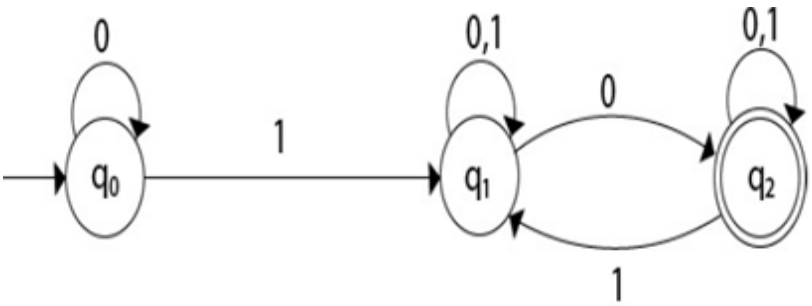
	ii. What type of derivation is followed by top-down and bottom-up parsers? iii. What is the purpose of a handle in shift-reduce parsing? iv. Every step of bottom-up parsing reduces a handle to its corresponding non terminal. Is it true or false? v. Why is left recursion a problem for top-down parsers like LL(1)? What is the difference between LR(0) and LR(1) items in parsers?				
2.a	Under what conditions does a SLR (1) parser encounter shift/reduce and reduce/reduce conflicts? For the grammar given below, apply the SLR (1) parsing algorithm and construct the corresponding DFA. Identify the state that causes a parsing conflict. S→L=R| R L→+R | num R→L	8	CO3	L3	3.2.2
b.	Consider the following CFG: lexp→atom | list atom →number | identifier list→(lexp –seq) lexp –seq → lexp -seq | lexp i. Remove the left recursion. ii. Construct First and Follow sets for the nonterminals of the resulting grammar.	6	CO3	L3	1.1.3
c.	What is shift and reduce action in bottom-up parser ?Describe how a stack is used to parse input strings in both top-down and bottom-up parsing approaches.	6	CO3	L2	1.1.3
3.a	Apply LALR(1) for the following grammar: B→(B) B→b Construct corresponding DFA and parse table	8	CO3	L3	3.2.2
b.	Perform left factoring on the following CFGs. i. A → aAB | aBc | aAc ii. S → S#cS | SS | S@| a | b | c iii. S → iEtS | iEtSeS | a iv. E → b	6	CO3	L3	3.2.2
c.	How can lookahead symbol be determined for a given production in an LR (1) parser? Illustrate with examples.	6	CO3	L2	1.1.3

Model Question Paper for End Semester Examination (ESA)		
Course Code: 25ECSC203		Course Title: Principles of Compiler Design
Duration:3 hrs		
Max. Marks:100		

Note: Answer five questions; any two full questions from each unit-I and unit-II and one full question from unit-III

UNIT 1

Q.No	Questions	Marks	CO	BL	PI Code
Q1.a	Design a regular expression for remote file identifiers using following constraints: i. Remote file identifiers have the format like user@hostname: filename. ii. The part of the identifiers made up of words, which is sequence of one or more letters and digits. iii. The user part contains a word. iv. A hostname consists of one or more words separated by periods (www.uio.no). v. Filename consists of one or more words separated by slash characters (/) with an optimal leading and /or trailing slash. The "user@" part is optional and may be omitted. The entire "user@hostname:" may also be omitted, including the trailing colon ":".	8	CO2	L3	1.1.3
b.	How do you use Thompson construction method to convert regular **(a+b)ab** to ϵ-NFA?	6	CO2	L3	1.1.3
c.	What is the role of finite automata in translating higher level language to machine level by compiler? Explain	6	CO1	L2	1.3.2
Q2.a	Minimize the following DFA :  	8	CO3	L3	1.1.3
b.	Let G=(V,T,P,S), where P is: S→S(E)|E E→(S)E|0|1|ϵ Show that the grammar is ambiguous	6	CO4	L3	3.2.2
c.	Compare the following: i. NFA and DFA	6	CO1	L2	1.1.3

	ii. Regular language and Context free language Syntax analysis and Semantic analysis				
Q3.a	Convert the following NFA to DFA using subset construction method.  	8	CO3	L3	1.1.3
b.	How do you use Chomsky hierarchy to classify the languages accepted by different machines? Explain.	6	CO1	L2	1.1.3
c.	Design the regular expression correspond to following grammar S→AB | AS, A→aA | a , B→b	6	CO2	L3	3.2.2
	UNIT 2				
Q4.a	Consider the grammar: S→L = R | R L→*R | id R → L (i) Construct set of LR(0) items (ii) Design DFA of LR(0) (iii) Show the given grammar is not SLR(1)	8	CO3	L3	3.2.2
b.	What is meant by handle processing? How it helps in shift reduce parsing? Explain with example	6	CO3	L2	1.1.3
c.	What is the need to removing left recursion in CFG before applying parsing algorithm? Eliminate Left Recursion from the grammar given below: S →(L) | x L → L, S | S	6	CO1	L3	1.1.3
Q5.a	Consider the grammar G={V,T,S,P} IF_STMT → **if** E **then** STMT | **if** E **then** STMT **else** STMT E → 0 | 1 STMT → s Can LL(1) technique parse any given input using the above grammar? If not, convert the grammar suitable for LL (1) parsing otherwise show the parsing for a valid input.	8	CO3	L3	3.2.2
b.	Compare and contrast SLR, LR and LALR algorithms.	6	CO3	L2	1.1.3
c.	Compute first and follow sets for the grammar G : S →A | C | other A→ id := E C → id (L)	6	CO2	L3	1.1.3

<table>
<tr><td></td><td>L → id , L | id</td><td></td><td></td><td></td><td></td></tr>
<tr><td>Q6.a.</td><td>Consider the following grammar
E → (L) | a
L → E L | E
Construct DFA of LR(1) items and show the parsing stack and actions of shift reduce parser for the input ((a) a (a a)) .</td><td>8</td><td>CO3</td><td>L3</td><td>3.2.2</td></tr>
<tr><td>b.</td><td>Use the error recovery process and fill the empty cells in the table given bellow:
E → T X   X → + E | ε
T → (E) | int Y   Y → * T | ε
<table>
<tr><td></td><td>int</td><td>*</td><td>+</td><td>(</td><td>)</td><td>$</td></tr>
<tr><td>T</td><td>int Y</td><td></td><td></td><td>(E)</td><td></td><td></td></tr>
<tr><td>E</td><td>T X</td><td></td><td></td><td>T X</td><td></td><td></td></tr>
<tr><td>X</td><td></td><td></td><td>+ E</td><td></td><td>ε</td><td>ε</td></tr>
<tr><td>Y</td><td></td><td>* T</td><td>ε</td><td></td><td>ε</td><td>ε</td></tr>
</table></td><td>6</td><td>CO3</td><td>L2</td><td>3.2.2</td></tr>
<tr><td>c.</td><td>How do you remove left factoring? Perform left factoring for the below given grammar.
S →aSSbS | aSaSb | abb |b</td><td>6</td><td>CO2</td><td>L3</td><td>1.1.3</td></tr>
<tr><td colspan="6">UNIT 3</td></tr>
<tr><td>Q7.a</td><td>What are S-attributes and L-attributes? Consider the following grammar :
Decl → Type Var_list
Type→int | float
Var_list →id , Var_list | id
Define Semantic rules (ii) Draw the dependency graph</td><td>8</td><td>CO4</td><td>L3</td><td>1.3.2</td></tr>
<tr><td>b.</td><td>Explain the algorithm used for attribute computation</td><td>6</td><td>CO4</td><td>L2</td><td>1.3.2</td></tr>
<tr><td>c.</td><td>Answer the followings
i. What type of information is stored in symbol table? List the operation performed on symbol table.
ii. What are major data structures and inherited attributes in Symbol table?</td><td>6</td><td>CO4</td><td>L2</td><td>1.3.2</td></tr>
<tr><td>Q8.a</td><td>Write quadruples and P code for the following C code fragment :

prod=1;
scanf(“%d”%d”,&a,&b);
while(a!=0)
{
prod=prod +b;
a=a-1;
}
printf(“%d”,prod);</td><td>8</td><td>CO4</td><td>L3</td><td>1.3.2</td></tr>
<tr><td>b.</td><td>Explain the translation of expression address code of Triples with example.</td><td>6</td><td>CO4</td><td>L2</td><td>1.3.2</td></tr>
</table>

c.	Generate the code for the following expression using 3-address code generator algorithm: x=(a + b)	6	CO4	L2	1.3.2